

MIT Art Design and Technology University
MIT School of Computing, Pune



21BTCS033-NLP
Class – L.Y. (SEM-I), <Core>

Unit - II Syntactic analysis

Dr. Saleha Saudagar

AY 2026-27 SEM-I



Unit II - Syllabus

Unit II – Syntactic analysis

Part of Speech (POS) tagging: rule based and stochastic tagging, stop words, sequence labelling: HMM, Conditional Random Field (CRF), Context Free Grammars (CFG), Parsing: top down, Earley parsing, and CKY parsing, Probabilistic Context Free Grammars (PCFG), Term Frequency and Inverse Document Frequency (TF-IDF), information extraction, relation extraction.

Reference Book: Speech and Language Processing: An introduction to natural language processing, computational linguistics, and speech recognition. By Daniel Jurafsky & James H. Martin.

What is syntactic analysis?

- Syntactic analysis is defined as analysis that tells us the logical meaning of certain given sentences or parts of those sentences.
- We also need to consider rules of grammar in order to define the logical meaning as well as correctness of the sentences.
- I have a sentence such as **School go a boy**, this sentence does not logically convey its meaning, and its grammatical structure is not correct.
- So, syntactic analysis tells us whether the given sentence conveys its logical meaning and whether its grammatical structure is correct.

- Syntactic analysis is a well-developed area of NLP that deals with the syntax of NL.
- In syntactic analysis, grammar rules have been used.

Parts of Speech

- From the earliest linguistic traditions (Yaska and Panini 5th C. BCE, Aristotle 4th C. BCE), the idea that words can be classified into grammatical categories
 - part of speech, word classes, POS, POS tags
- 8 parts of speech attributed to Dionysius Thrax of Alexandria (c. 1st C. BCE):
 - noun, verb, pronoun, preposition, adverb, conjunction, participle, article
- These categories are relevant for NLP today.

Two classes of words: Open vs. Closed

- Closed class words
 - Relatively fixed membership
 - Usually **function** words: short, frequent words with grammatical function
 - determiners: *a, an, the*
 - pronouns: *she, he, I*
 - prepositions: *on, under, over, near, by, ...*
- Open class words
 - Usually **content** words: Nouns, Verbs, Adjectives, Adverbs
 - Plus interjections: *oh, ouch, uh-huh, yes, hello*
 - New nouns and verbs like *iPhone* or *to fax*

Open class ("content")

Nouns

Proper

Janet
Italy

Common

cat, cats
mango

Verbs

Main

eat
went

Adjectives *old green tasty*

Adverbs *slowly yesterday*

Numbers

122,312
one

Interjections *Ow hello*

... more

Closed class ("function")

Determiners *the some*

Conjunctions *and or*

Pronouns *they its*

Auxiliary

can
had

Prepositions *to with*

Particles *off up*

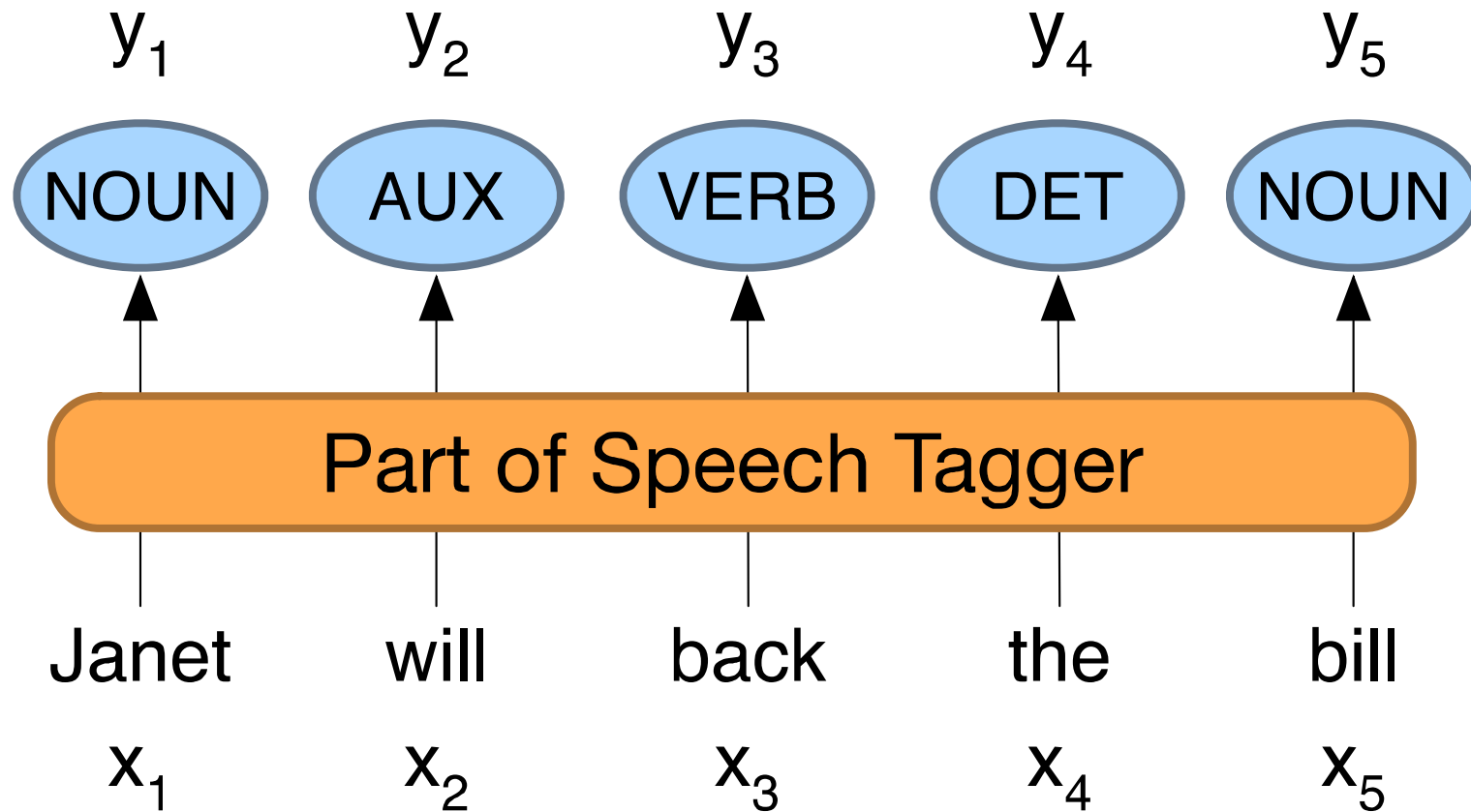
... more

Part-of-Speech Tagging

- Assigning a part-of-speech to each word in a text.
- Words often have more than one POS.
- **book:**
 - VERB: (***Book** that flight*)
 - NOUN: (*Hand me that **book***).

Part-of-Speech Tagging

Map from sequence $x_1, \dots, x_n$ of words to $y_1, \dots, y_n$ of POS tags



"Universal Dependencies" Tagset

	Tag	Description	Example
Open Class	ADJ	Adjective: noun modifiers describing properties	<i>red, young, awesome</i>
	ADV	Adverb: verb modifiers of time, place, manner	<i>very, slowly, home, yesterday</i>
	NOUN	words for persons, places, things, etc.	<i>algorithm, cat, mango, beauty</i>
	VERB	words for actions and processes	<i>draw, provide, go</i>
	PROPN	Proper noun: name of a person, organization, place, etc..	<i>Regina, IBM, Colorado</i>
	INTJ	Interjection: exclamation, greeting, yes/no response, etc.	<i>oh, um, yes, hello</i>
Closed Class Words	ADP	Adposition (Preposition/Postposition): marks a noun's spacial, temporal, or other relation	<i>in, on, by under</i>
	AUX	Auxiliary: helping verb marking tense, aspect, mood, etc.,	<i>can, may, should, are</i>
	CCONJ	Coordinating Conjunction: joins two phrases/clauses	<i>and, or, but</i>
	DET	Determiner: marks noun phrase properties	<i>a, an, the, this</i>
	NUM	Numeral	<i>one, two, first, second</i>
	PART	Particle: a preposition-like form used together with a verb	<i>up, down, on, off, in, out, at, by</i>
	PRON	Pronoun: a shorthand for referring to an entity or event	<i>she, who, I, others</i>
	SCONJ	Subordinating Conjunction: joins a main clause with a subordinate clause such as a sentential complement	<i>that, which</i>
Other	PUNCT	Punctuation	<i>; , ()</i>
	SYM	Symbols like \$ or emoji	<i>\$, %</i>
	X	Other	<i>asdf, qwfg</i>

Sample "Tagged" English sentences

- There/**PRO** were/**VERB** 70/**NUM** children/**NOUN** there/**ADV** ./**PUNC**
- Preliminary/**ADJ** findings/**NOUN** were/**AUX** reported/**VERB** in/**ADP**
today/**NOUN** 's/**PART** New/**PROPN** England/**PROPN** Journal/**PROPN**
of/**ADP** Medicine/**PROPN**

Why Part of Speech Tagging?

- Can be useful for other NLP tasks
 - Parsing: POS tagging can improve syntactic parsing
 - MT: reordering of adjectives and nouns (say from Spanish to English)
 - Sentiment or affective tasks: may want to distinguish adjectives or other POS
 - Text-to-speech (how do we pronounce “lead” or “object”?)
- Or linguistic or language-analytic computational tasks
 - Need to control for POS when studying linguistic change like creation of new words, or meaning shift
 - Or control for POS in measuring meaning similarity or difference

Standard algorithms for POS tagging



- Supervised Machine Learning Algorithms:
 - Hidden Markov Models
 - Conditional Random Fields (CRF)/ Maximum Entropy Markov Models (MEMM)
 - Neural sequence models (RNNs or Transformers)
 - Large Language Models (like BERT), finetuned
- All required a hand-labeled training set, all about equal performance (97% on English)

RULE-BASED PART-OF-SPEECH TAGGING



Earlier POS tagging were a two-stage architecture

- **First stage** — In the first stage, it uses a dictionary to assign each word a list of potential parts-of-speech.
- **Second stage** – In the second stage, it uses large lists of hand-written disambiguation rules to sort down the list to a single part-of-speech for each word.

RULE-BASED PART-OF-SPEECH TAGGING



- One of the oldest techniques of tagging is rule-based POS tagging.
- Rule-based taggers use dictionary or lexicon for getting possible tags for tagging each word.
- If the word has more than one possible tag, then rule-based taggers use hand-written rules to identify the correct tag.

Here is an example of how a rule-based POS tagger might work:

1. Define a set of rules for assigning POS tags to words.

For example:

- If the word ends in “-tion,” assign the tag “noun.”
- If the word ends in “-ment,” assign the tag “noun.”
- If the word is all uppercase, assign the tag “proper noun.”
- If the word is a verb ending in “-ing,” assign the tag “verb.”

Iterate through the words in the text and apply the rules to each word in turn. For example:

- “Nation” would be tagged as “noun” based on the first rule.
- “Investment” would be tagged as “noun” based on the second rule.
- “UNITED” would be tagged as “proper noun” based on the third rule.

This is a very basic example of a rule-based POS tagger, and more complex systems can include additional rules and logic to handle more varied text

Stochastic tagging



- Another technique of tagging is Stochastic POS Tagging.
- Now, the question that arises here is which model can be stochastic.
- The model that includes frequency or probability (statistics) can be called stochastic.
- In statistical POS tagging, a model is trained on a large annotated corpus of text to learn the patterns and characteristics of different parts of speech.
- The model uses this training data to predict the POS tag of a given word based on the context in which it appears and the probability of different POS tags occurring in that context.

The simplest stochastic tagger applies the following approaches for POS tagging.

1. Word Frequency Approach

the tag encountered most frequently with the word in the training set is the one assigned to an ambiguous instance of that word. The main issue with this approach is that it may yield inadmissible sequence of tags.

2. Tag Sequence Probabilities

tagger calculates the probability of a given sequence of tags occurring. It is also called n-gram approach. It is called so because the best tag for a given word is determined by the probability at which it occurs with the n previous tags

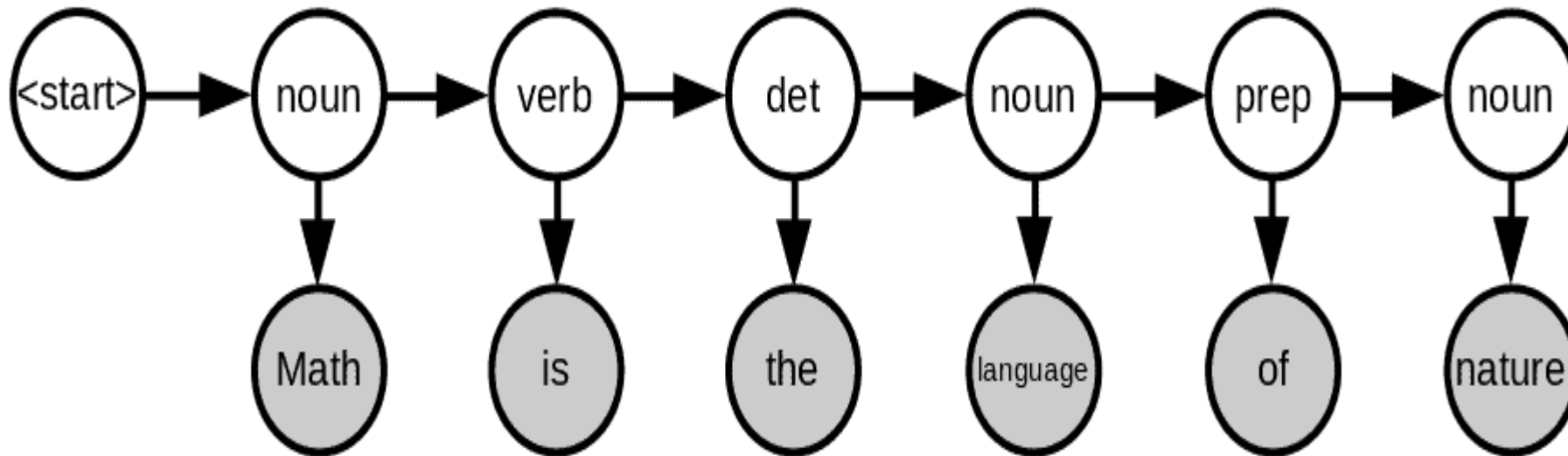
Hidden Markov Models (HMMs)



- Hidden Markov models (HMMs) are a type of statistical model that can be used for part-of-speech (POS) tagging in natural language processing (NLP).
- In an HMM-based POS tagger, a model is trained on a large annotated corpus of text to learn the patterns and characteristics of different parts of speech.
- The model uses this training data to predict the POS tag of a given word based on the probability of different tags occurring in the context of the word.

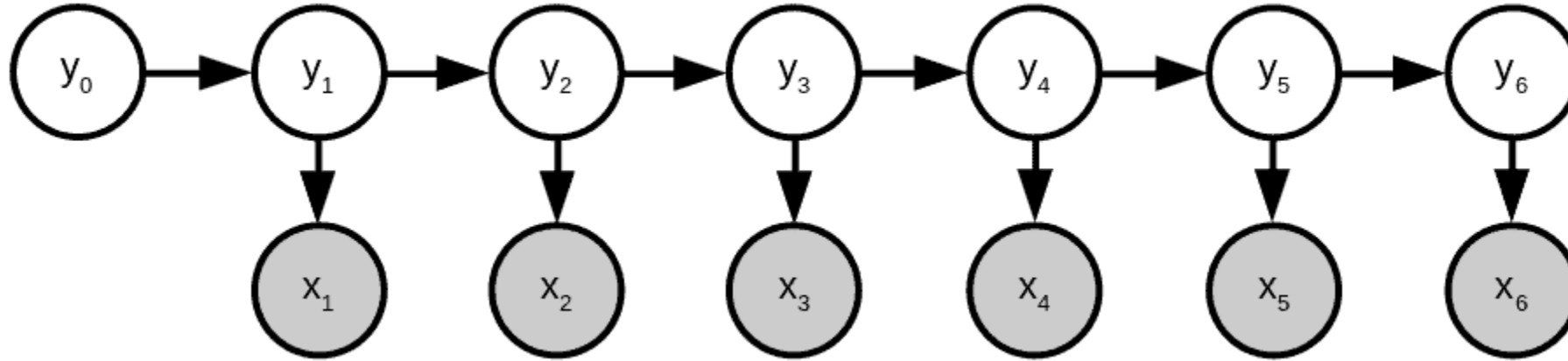
- An HMM-based POS tagger consists of a set of states, each corresponding to a possible POS tag, and a set of transitions between the states.
- The model is trained on the training data to learn the probabilities of transitioning from one state to another and the probabilities of observing different words given a particular state.

we have an English sentence like Math is the language of nature. We want to label each word with its part of speech. That way, we get a graph:



This graph represents a first-order hidden Markov model.

The general first-order HMM graph for any sequence is similar to that of our example:



We can assign probabilities to each arrow and model the occurrence of such a sequence using the product of these probabilities:

$$p(\mathbf{x}, \mathbf{y}) = \prod_{i=0}^{i=n} p(x_i | y_i) p(y_i | y_{i-1})$$

where $\mathbf{x}$ and $\mathbf{y}$ are the sequence and the array of its labels. The $p(x_i | y_i)$ are emission probabilities and the $p(y_i | y_{i-1})$ are transition probabilities.

- The **transition probabilities** describe the probability of transitioning from one hidden state to another.
- The **emission probabilities** describe the probability of observing an output given a hidden state.

Conditional Random Field (CRF)



Named Entities

- **Named entity**, in its core usage, means anything that can be referred to with a proper name. Most common 4 tags:
 - **PER** (Person): “Marie Curie”
 - **LOC** (Location): “New York City”
 - **ORG** (Organization): “Stanford University”
 - **GPE** (Geo-Political Entity): “Boulder, Colorado”
- Often multi-word phrases
- But the term is also extended to things that aren't entities:
 - dates, times, prices

Named Entity tagging

- The task of named entity recognition (NER):
 - find spans of text that constitute proper names
 - tag the type of the entity.

Why NER?

- Sentiment analysis: consumer's sentiment toward a particular company or person?
- Question Answering: answer questions about an entity?
- Information Extraction: Extracting facts about entities from text.

Conditional Random Fields

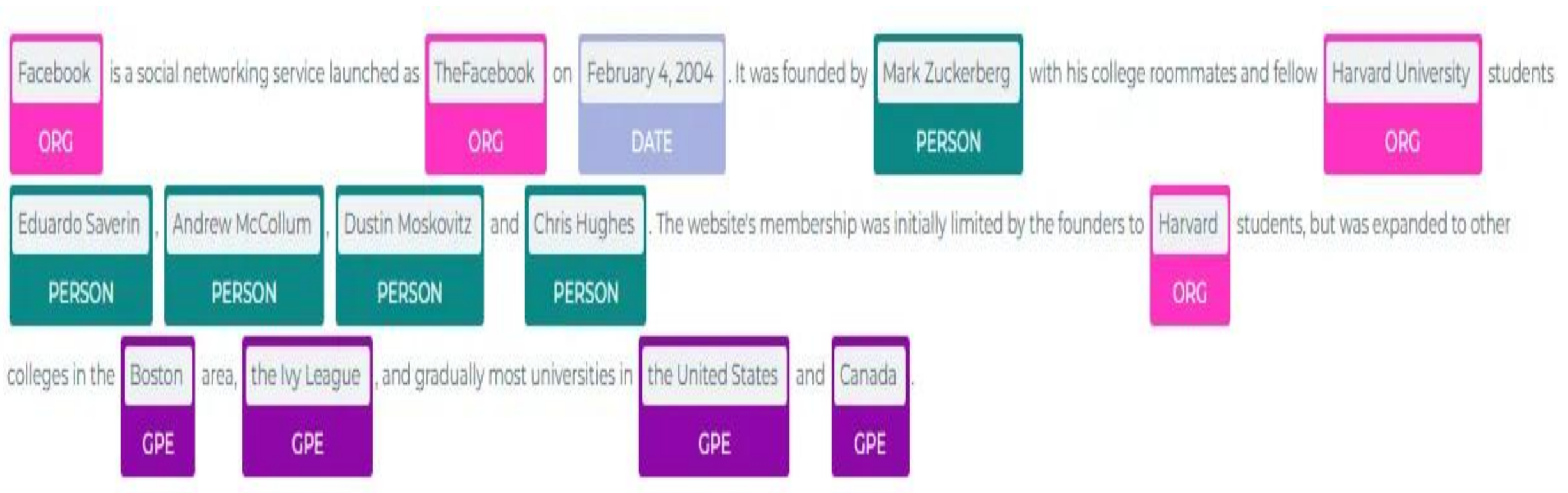
- Conditional Random Fields (CRFs) are a class of probabilistic graphical model that is commonly used in machine learning and natural language processing applications.
- In NLP, CRFs are used for the **sequence labeling** tasks, which involve assigning labels to each element in a sequence of observations, such as assigning part-of-speech tags to words in a sentence or recognizing named entities (such as people, organizations, and locations) in a text.
- CRFs use the observed data to predict the labels of the sequence, while taking into account the dependencies between neighboring labels.

CRF model for NER is trained on a labeled dataset that includes examples of text with corresponding named entity labels.

During training, the model learns to identify patterns and features in the input text that are associated with named entities, such as the presence of specific words or phrases, syntactic structures, or contextual information.

Once the model is trained, it can be used to **predict named entities** by assigning labels to each token based on the learned patterns and features.

The figure below shows the visualization of the named entities recognized from a sample text. The entities are extracted, labelled (as PERSON, DATE, ORG etc) and displayed on the original text.



Below is the formula for CRF where Y is the hidden state (for example, part of speech) and X is the observed variable

$$p(\mathbf{y}|\mathbf{x}) = \underbrace{\frac{1}{Z(\mathbf{x})}}_{\text{Normalization}} \prod_{t=1}^T \exp \left\{ \sum_{k=1}^K \underbrace{\theta_k}_{\text{Weight}} \underbrace{f_k(y_t, y_{t-1}, \mathbf{x}_t)}_{\text{Feature}} \right\}$$

Advantages of CRF

Why CRF became the go-to model for NLP sequence labeling

High Accuracy

CRF achieves state-of-the-art results on POS tagging, NER, and chunking benchmarks due to global sequence optimisation.

Rich Overlapping Features

Unlike HMM, CRF can use any number of features simultaneously — word shape, context, suffix, POS hints — all at once.

Full Sequence Consideration

CRF jointly optimises the entire output sequence rather than making greedy local decisions at each step.

Label Consistency

Avoids invalid tag sequences (e.g., I-PER after O) by penalising illegal transitions during training.

Flexible Architecture

Works standalone or as the output layer in neural models (BiLSTM-CRF, BERT-CRF) for even higher accuracy.

Handles Any Feature

Can incorporate domain knowledge, gazetteers, regex patterns, and distributional embeddings as features.

Activate Windows

CRF vs HMM

Why CRF outperforms Hidden Markov Models

Feature	HMM	CRF
Model Type	Generative	Discriminative
What it models	$P(X, Y)$ — Joint	$P(Y X)$ — Conditional
Feature Richness	Limited	Rich / Overlapping
Sequence Optimisation	Per-step (greedy)	Full sequence jointly
Accuracy (NLP tasks)	Moderate	High
Independence Assumption	Strong (Naive)	Relaxed
Feature Engineering	Very Limited	Flexible & Powerful

✓ **VERDICT:** CRF removes HMM's strong independence assumption and can use any number of rich, overlapping features.

Activate Windows

Go to Settings to activate Windows.

Features Used in CRF

Rich, overlapping feature templates — the power behind CRF

Current Word

The word being labeled right now

Previous Word

The word that appeared before current word

Next Word

The word that follows current word

Capitalisation

Is the word Title Case? ALL CAPS? mixed?

Prefix / Suffix

First/last 2-3 letters (e.g., -ing, -tion, Un-)

Contains Digit

Does the token have numbers? (e.g., 2024, \$100)

Previous Tag

What tag was assigned to the previous token?

Word Shape

Pattern like XxxX (TitleCase) or XXXX (AllCaps)

 **Example:** Word = "Mumbai" → starts-with-capital ✓ | known-city ✓ | after-preposition-"in" ✓ ⇒ **Label = LOCATION**

Limitations of CRF

Important trade-offs to understand

Slow Training

Optimising CRF parameters requires computing partition functions (normalisation), which is computationally expensive for large datasets.

Feature Engineering

Traditional CRF requires hand-crafted feature templates. This demands significant domain expertise and is task-specific.

Limited Long-Range Context

Linear-chain CRF captures only first-order Markov dependencies (previous tag). Very long-range dependencies require extensions.

Requires Labeled Data

CRF is fully supervised. High-quality, annotated training data is mandatory — which is expensive and time-consuming to create.

Memory Intensive

Large feature spaces (millions of features) require significant memory. Scaling to large corpora needs careful engineering.

Superseded in Some Areas

For tasks with massive data, end-to-end neural models (BERT fine-tuning) sometimes outperform even BiLSTM-CRF without feature engineering.

Activate Windows

Context Free Grammars (CFG)



CONSTITUENCY

- The word syntax comes from the Greek syntaxis ' , meaning “setting out together or arrangement”, and refers to the way words are arranged together.
- The fundamental idea of constituency is that groups of words may behave as a single unit or phrase, called a constituent.
- For example we will see that a group of words called a noun phrase often acts as a unit; noun phrases include single words like she or Michael and phrases like the house, Russian Hill

- The most commonly used mathematical system for modeling constituent structure in English and other natural languages is the Context-Free Grammar, or CFG.
- CFG are also called as Phrase-Structure Grammars, and the formalism is equivalent to what is also called Backus-Naur Form or BNF.
- A context-free grammar consists of a set of rules or productions, each of which expresses the ways that symbols of the language can be grouped and ordered together, and a lexicon of words and symbols.

Context-free grammar

$$G = \langle T, N, S, R \rangle$$

- T is set of terminals (lexicon)
- N is set of non-terminals For NLP, we usually distinguish out a set
- $P \subset N$ of *preterminals* which always rewrite as terminals.
- S is start symbol (one of the nonterminals)
- R is rules/productions of the form $X \rightarrow \gamma$, where X is a nonterminal and γ is a sequence of terminals and nonterminals (may be empty).
- A grammar G generates a language L .

For e.g :

NP → Det Nominal

NP → ProperNoun

Nominal → Noun | Nominal Noun

NP (or noun phrase), can be composed of either a ProperNoun or a determiner (Det) followed by a Nominal; a Nominal can be one or more Nouns.

The symbols that are used in a CFG are divided into two classes.

- The symbols that correspond to words in the language (“the”, “nightclub”) are called terminal symbols. The lexicon is the set of rules that introduce these terminal symbols.
- The symbols that express clusters or generalizations of these are called non-terminals.
- In each context free rule, the item to the right of the arrow ($\rightarrow$) is an ordered list of one or more terminals and non-terminals, while to the left of the arrow is a single non-terminal symbol expressing some cluster or generalization

An example context-free grammar

$G = \langle T, N, S, R \rangle$

$T = \{that, this, a, the, man, book, flight, meal, include, read, does\}$

$N = \{S, NP, NOM, VP, Det, Noun, Verb, Aux\}$

$S = S$

$R = \{$

$S \rightarrow NP VP$

$S \rightarrow Aux NP VP$

$S \rightarrow VP$

$NP \rightarrow Det NOM$

$NOM \rightarrow Noun$

$NOM \rightarrow Noun NOM$

$VP \rightarrow Verb$

$VP \rightarrow Verb NP$

$Det \rightarrow that \mid this \mid a \mid the$

$Noun \rightarrow book \mid flight \mid meal \mid man$

$Verb \rightarrow book \mid include \mid read$

$Aux \rightarrow does$

$\}$

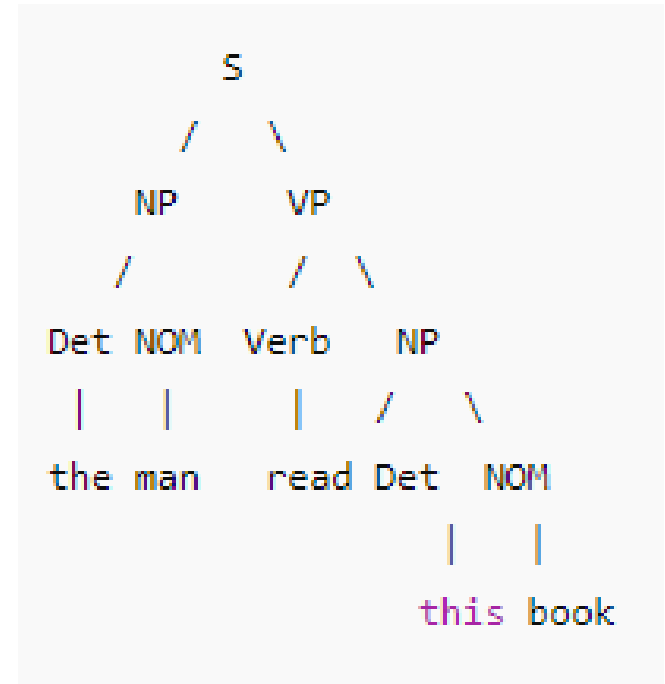
Application of grammar rewrite rules

$S \rightarrow NP VP$	$Det \rightarrow that \mid this \mid a \mid the$
$S \rightarrow Aux NP VP$	$Noun \rightarrow book \mid flight \mid meal \mid man$
$S \rightarrow VP$	$Verb \rightarrow book \mid include \mid read$
$NP \rightarrow Det NOM$	$Aux \rightarrow does$
$NOM \rightarrow Noun$	
$NOM \rightarrow Noun NOM$	
$VP \rightarrow Verb$	
$VP \rightarrow Verb NP$	

Application of grammar rewrite rules

$S \rightarrow NP VP$	$Det \rightarrow that \mid this \mid a \mid the$
$S \rightarrow Aux NP VP$	$Noun \rightarrow book \mid flight \mid meal \mid man$
$S \rightarrow VP$	$Verb \rightarrow book \mid include \mid read$
$NP \rightarrow Det NOM$	$Aux \rightarrow does$
$NOM \rightarrow Noun$	
$NOM \rightarrow Noun NOM$	
$VP \rightarrow Verb$	
$VP \rightarrow Verb NP$	

$S \rightarrow NP VP$
 $\rightarrow Det NOM VP$
 $\rightarrow The NOM VP$
 $\rightarrow The Noun VP$
 $\rightarrow The man VP$
 $\rightarrow The man Verb NP$
 $\rightarrow The man read NP$
 $\rightarrow The man read Det NOM$
 $\rightarrow The man read this NOM$
 $\rightarrow The man read this Noun$
 $\rightarrow The man read this book$



Example context-free grammar

$G = (V, T, S, P)$

$V = \{S, NP, VP, PP, Det, Noun, Verb, Aux, Pre\}$

$T = \{'a', 'ate', 'cake', 'child', 'fork', 'the', 'with'\}$

$S = S$

$P = \{ S \rightarrow NP VP$

$NP \rightarrow Det Noun \mid NP PP$

$PP \rightarrow Pre NP$

$VP \rightarrow Verb NP$

$Det \rightarrow 'a' \mid 'the'$

$Noun \rightarrow 'cake' \mid 'child' \mid 'fork'$

$Pre \rightarrow 'with'$

$Verb \rightarrow 'ate'\}$

(NP – Noun Phrase, VP – Verb Phrase, PP – Prepositional Phrase, Det – Determiner, Aux – Auxiliary verb)

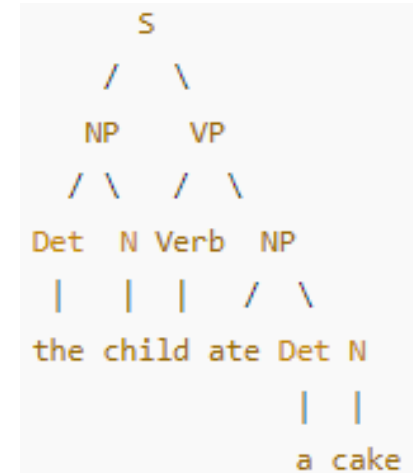
1. Check that 'the child ate a cake' is valid or not.
2. 'The child ate a cake with fork' is valid or not.
3. 'The child cake ate.

```

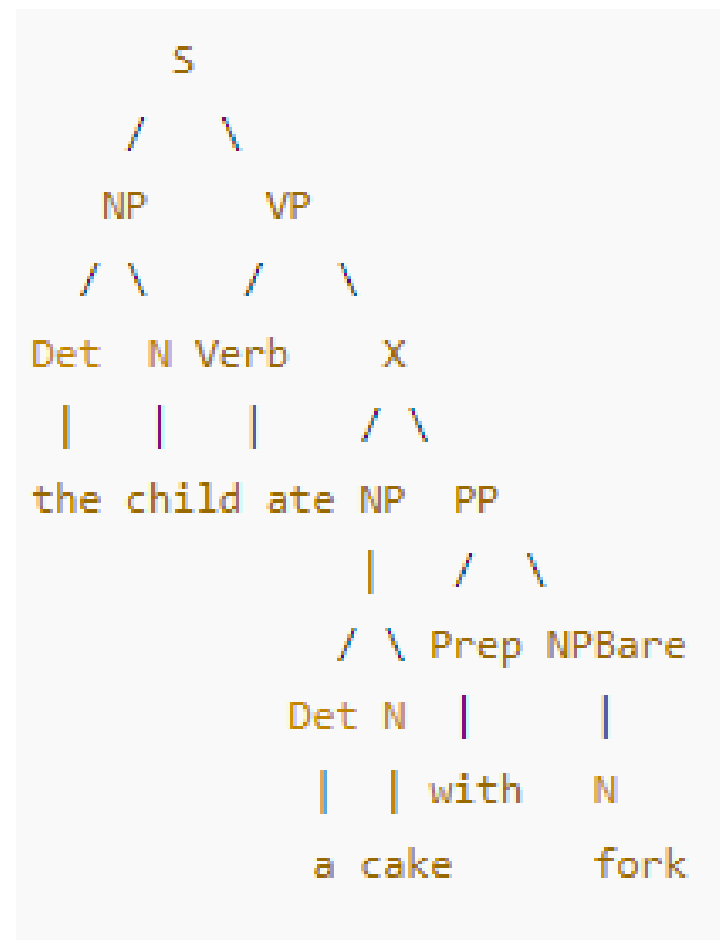
S → NP VP
NP → Det N
VP → Verb NP
Det → the | a
N → child | cake
Verb → ate
  
```

```

S
⇒ NP VP
⇒ (Det N) VP
⇒ the child VP
⇒ the child (Verb NP)
⇒ the child ate NP
⇒ the child ate (Det N)
⇒ the child ate a cake
  
```



S
 ⇒ NP VP
 ⇒ (Det N) VP
 ⇒ the child VP
 ⇒ the child (Verb NP PP)
 ⇒ the child ate NP PP
 ⇒ the child ate (Det N) PP
 ⇒ the child ate a cake PP
 ⇒ the child ate a cake (Prep NPBare)
 ⇒ the child ate a cake with NPBare
 ⇒ the child ate a cake with N
 ⇒ the child ate a cake with fork



Parsing: top down, Earley parsing, and CKY parsing

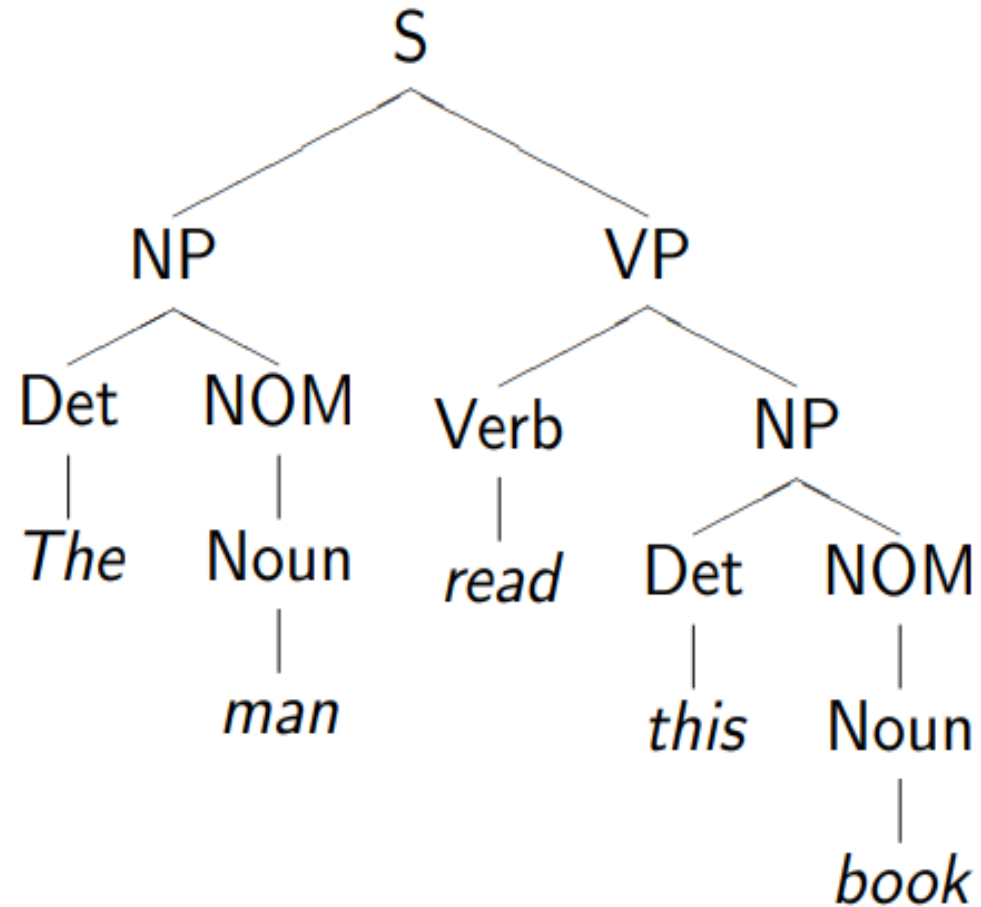


- The kind of structures assigned by CFG.
- Automatically assigning a complete context-free (phrase structure) tree to an input sentence.
- Parse trees are useful in applications such as grammar checking in word-processing systems.
- A sentence which cannot be parsed may have grammatical errors.

What is parsing?

- We want to run the grammar backwards to find the structure.
- Parsing can be viewed as a search problem.
- We search through the legal rewritings of the grammar.
- We want to find *all* structures matching an input string of words (for the moment)
- We can do this bottom-up or top-down
- This distinction is independent of depth-first versus breadth-first; we can do either both ways.
- Doing this we build a *search tree* which is different from the *parse tree*.

Parse tree for above example



Top-down parsing

- Top-down parsing is goal-directed.
- A top-down parser starts with a list of constituents to be built.
- It rewrites the goals in the goal list by matching one against the LHS of the grammar rules, and expanding it with the RHS.
- Attempting to match the sentence to be derived.
- If a goal can be rewritten in several ways, then there is a choice of which rule to apply (search problem)
- Can use depth-first or breadth-first search, and goal ordering.

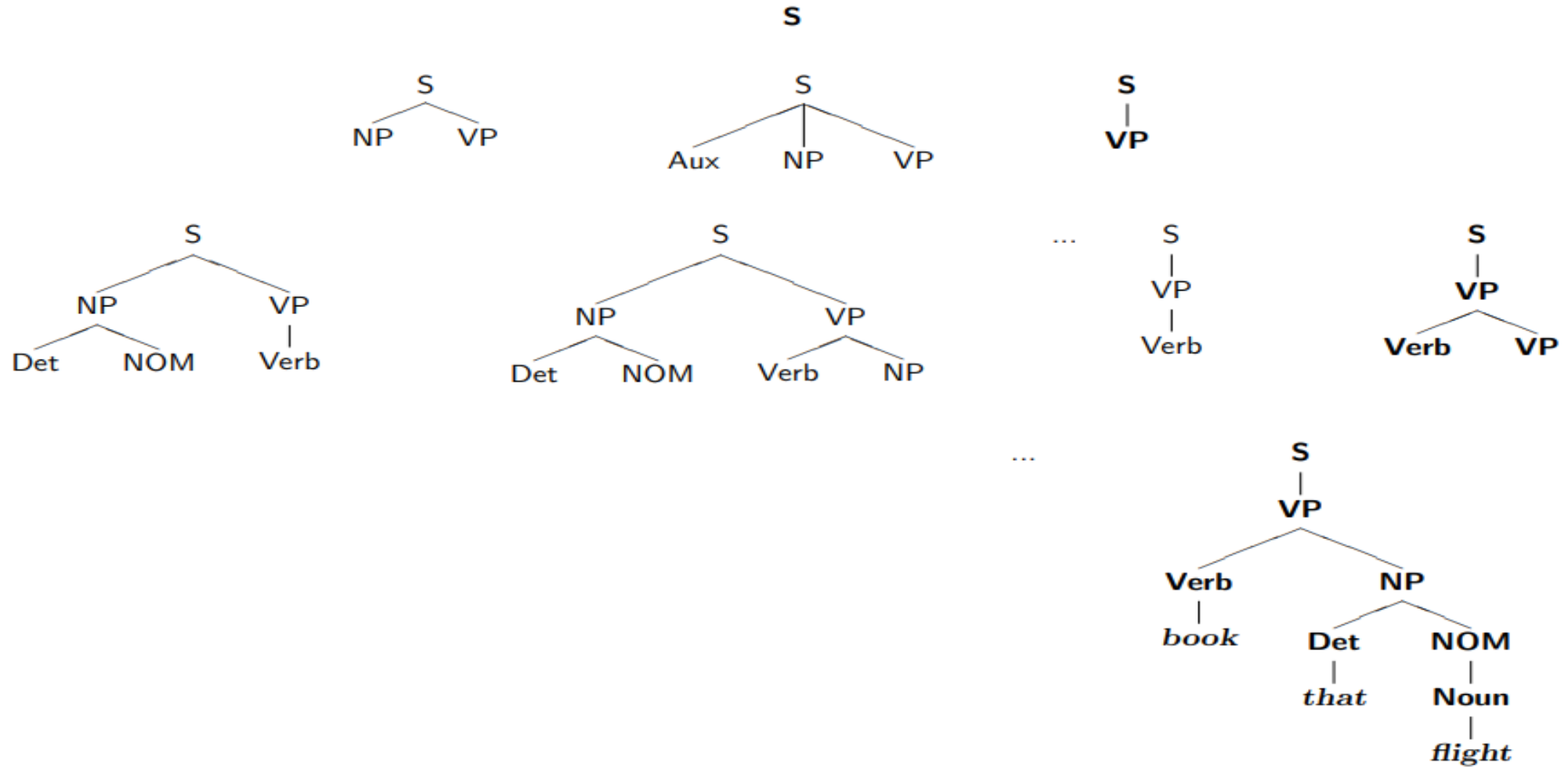
Top-down parsing example (Breadth-first)

$S \rightarrow NP VP$	$Det \rightarrow that \mid this \mid a \mid the$
$S \rightarrow Aux NP VP$	$Noun \rightarrow book \mid flight \mid meal \mid man$
$S \rightarrow VP$	$Verb \rightarrow book \mid include \mid read$
$NP \rightarrow Det NOM$	$Aux \rightarrow does$
$NOM \rightarrow Noun$	
$NOM \rightarrow Noun NOM$	
$VP \rightarrow Verb$	
$VP \rightarrow Verb NP$	

Book that flight.

(Work out top-down, breadth-first search on the board...)

Top-down parsing example (Breadth-first)



Problems with top-down parsing

- Left recursive rules... e.g. $NP \rightarrow NP PP...$ lead to infinite recursion
- Will do badly if there are many different rules for the same LHS. Consider if there are 600 rules for S , 599 of which start with NP , but one of which starts with a V , and the sentence starts with a V .
- Useless work: expands things that are possible top-down but not there (no bottom-up evidence for them).
- Top-down parsers do well if there is useful grammar-driven control: search is directed by the grammar.
- In practice that is always done bottom-up as lexical lookup.

Bottom-up parsing

- Top-down parsing is data-directed.
- The initial goal list of a bottom-up parser is the string to be parsed.
- If a sequence in the goal list matches the RHS of a rule, then this sequence may be replaced by the LHS of the rule.
- Parsing is finished when the goal list contains just the start symbol.
- If the RHS of several rules match the goal list, then there is a choice of which rule to apply (search problem)
- Can use depth-first or breadth-first search, and goal ordering.
- The standard presentation is as **shift-reduce** parsing.

Shift Reduce Parser

- Start with the sentence to be parsed in an input buffer.
- a "shift" action corresponds to pushing the next input symbol from the buffer onto the stack
- a "reduce" action occurs when we have a rule's RHS on top of the stack. To perform the reduction, we pop the rule's RHS off the stack and replace it with the terminal on the LHS of the corresponding rule.
- (When either "shift" or "reduce" is possible, choose one arbitrarily.)
- If you end up with only the START symbol on the stack, then success!
- If you don't, and you cannot and no "shift" or "reduce" actions are possible, backtrack.

Bottom-up parsing example

$S \rightarrow NP VP$	$Det \rightarrow that \mid this \mid a \mid the$
$S \rightarrow Aux NP VP$	$Noun \rightarrow book \mid flight \mid meal \mid man$
$S \rightarrow VP$	$Verb \rightarrow book \mid include \mid read$
$NP \rightarrow Det NOM$	$Aux \rightarrow does$
$NOM \rightarrow Noun$	
$NOM \rightarrow Noun NOM$	
$VP \rightarrow Verb$	
$VP \rightarrow Verb NP$	

Four main operations

1. **Shift** $\rightarrow$ Move (shift) the next input symbol from the input buffer to the stack.
2. **Reduce** $\rightarrow$ Replace a sequence of grammar symbols on the stack (that matches the RHS of a production) with the LHS non-terminal.
3. **Accept** $\rightarrow$ Parsing is successful (stack contains only the start symbol and input is empty).
4. **Error** $\rightarrow$ No valid moves; input is rejected.

Shift-reduce parsing

Stack	Input remaining	Action
()	Book that flight	shift
(Book)	that flight	reduce, Verb → book, (Choice #1 of 2)
(Verb)	that flight	shift
(Verb that)	flight	reduce, Det → that
(Verb Det)	flight	shift
(Verb Det flight)		reduce, Noun → flight
(Verb Det Noun)		reduce, NOM → Noun
(Verb Det NOM)		reduce, NP → Det NOM
(Verb NP)		reduce, VP → Verb NP
(VP)		reduce, S → VP
(S)		SUCCESS!

Ambiguity may lead to the need for backtracking.

Top Down vs. Bottom Up

- Top down never explores options that will not lead to a full parse, but can explore many options that never connect to the actual sentence.
- Bottom up never explores options that do not connect to the actual sentence but can explore options that can never lead to a full parse.
- Relative amounts of wasted search depend on how much the grammar branches in each direction.

CFGs can capture recursion

Example of seemingly endless recursion of embedded prepositional phrases:

PP $\rightarrow$ Prep NP

NP $\rightarrow$ Noun PP

[_S The mailman ate his [_{NP} lunch [_{PP} with his friend [_{PP} from the cleaning staff [_{PP} of the building [_{PP} at the intersection [_{PP} on the north end [_{PP} of town]]]]]]].

(Bracket notation)

Dynamic Programming Parsing

- To avoid extensive repeated work, must cache intermediate results, i.e. completed phrases.
- Caching (memorizing) critical to obtaining a polynomial time parsing (recognition) algorithm for CFGs.
- Dynamic programming algorithms based on both top-down and bottom-up search can achieve $O(n^3)$ recognition time where n is the length of the input string.

Dynamic Programming Parsing Methods



- **CKY** (Cocke-Kasami-Younger) algorithm based on bottom-up parsing and requires first normalizing the grammar.
- **Earley parser** is based on top-down parsing and does not require normalizing grammar but is more complex.
- More generally, **chart parsers** retain completed phrases in a chart and can combine top-down and bottom-up search.

CKY

- First grammar must be converted to **Chomsky normal form (CNF)** in which productions must have either exactly 2 non-terminal symbols on the RHS or 1 terminal symbol (lexicon rules).
- Parse bottom-up storing phrases formed from all substrings in a triangular table (chart).
- if we have a rule like

$$A \rightarrow B C \gamma$$

we replace the leftmost pair of non-terminals with a new non-terminal and introduce a new production result in the following new rules.

$$X1 \rightarrow B C \quad , \quad A \rightarrow X1 X2 \quad , \quad X2 \rightarrow \gamma$$

ATIS English Grammar Conversion

Original Grammar

$S \rightarrow NP VP$

$S \rightarrow Aux NP VP$

$S \rightarrow VP$

$NP \rightarrow Pronoun$

$NP \rightarrow Proper-Noun$

$NP \rightarrow Det Nominal$

$Nominal \rightarrow Noun$

$Nominal \rightarrow Nominal Noun$

$Nominal \rightarrow Nominal PP$

$VP \rightarrow Verb$

$VP \rightarrow Verb NP$

$VP \rightarrow VP PP$

$PP \rightarrow Prep NP$

Chomsky Normal Form

$S \rightarrow NP VP$

$S \rightarrow X1 VP$

$X1 \rightarrow Aux NP$

$S \rightarrow book \mid include \mid prefer$

$S \rightarrow Verb NP$

$S \rightarrow VP PP$

$NP \rightarrow I \mid he \mid she \mid me$

$NP \rightarrow Houston \mid NWA$

$NP \rightarrow Det Nominal$

$Nominal \rightarrow book \mid flight \mid meal \mid money$

$Nominal \rightarrow Nominal Noun$

$Nominal \rightarrow Nominal PP$

$VP \rightarrow book \mid include \mid prefer$

$VP \rightarrow Verb NP$

$VP \rightarrow VP PP$

$PP \rightarrow Prep NP$

CKY Parser

	Book	the	flight	through	Houston
	j= 1	2	3	4	5
i= 0					
1					
2					
3					
4					

**Cell[i,j]
contains all
constituents
(non-terminals)
covering words
 $i+1$ through j**

CKY Parser

Book the flight through Houston

0 Book 1 The 2 flight 3 through 4 houston 5

S, VP, Verb, Nominal, Noun	None			
	Det ←	NP		
		Nominal, Noun		

$$[0 \ 1] * [1 \ 2] =$$

$$[1 \ 2] * [2 \ 3] = \text{NP}$$

CKY Parser

Book the flight through Houston

S, VP, Verb, Nominal, Noun	None	VP		
	Det	NP		
		Nominal, Noun		

CKY Parser

Book the flight through Houston

S, VP, Verbs, Nominal, Noun	None	S VP		
	Det	NP		
		Nominal, Noun		

CKY Parser

Book the flight through Houston

S, VP, Verb, Nominal, Noun	None	S VP	None	
	Det	NP	None	
		Nominal, Noun	None	
			Prep	

$[2\ 3] * [3\ 4] = \text{None}$

CKY Parser

Book the flight through Houston

S, VP, Verb, Nominal, Noun	None	S VP	None	
	Det	NP	None	
		Nominal, Noun	None	
			Prep ← PP	
				NP ProperNoun

$[3\ 4] * [4\ 5] = PP$

CKY Parser

Book the flight through Houston

S, VP, Verb, Nominal, Noun	None	S VP	None	
	Det	NP	None	
		Nominal, Noun	None	Nominal
			Prep	PP
				NP ProperNoun

Diagram illustrating the CKY Parser parse tree for the sentence "Book the flight through Houston". The tree structure is shown as a grid of cells, where each cell contains a phrase structure rule (PSR) or a terminal word. Arrows indicate the derivation of the phrase "flight through Houston" from the PSR "Nominal" in the third row, fourth column.

CKY Parser

Book the flight through Houston

S, VP, Verb, Nominal, Noun	None	S VP	None	
	De←	NP	None	NP
		Nominal, Noun	None	Nominal
			Prep	PP
				NP ProperNoun

CKY Parser

Book the flight through Houston

S, VP, Verb, Nominal, Noun	None	S VP	None	VP
	Det	NP	None	NP
		Nominal, Noun	None	Nominal
			Prep	PP
				NP ProperNoun

CKY Parser

Book the flight through Houston

S, VP, Verb, Nominal, Noun	None	S VP	None	S VP
	Det	NP	None	NP
		Nominal, Noun	None	Nominal
			Prep	PP
				NP ProperNoun

CKY Parser

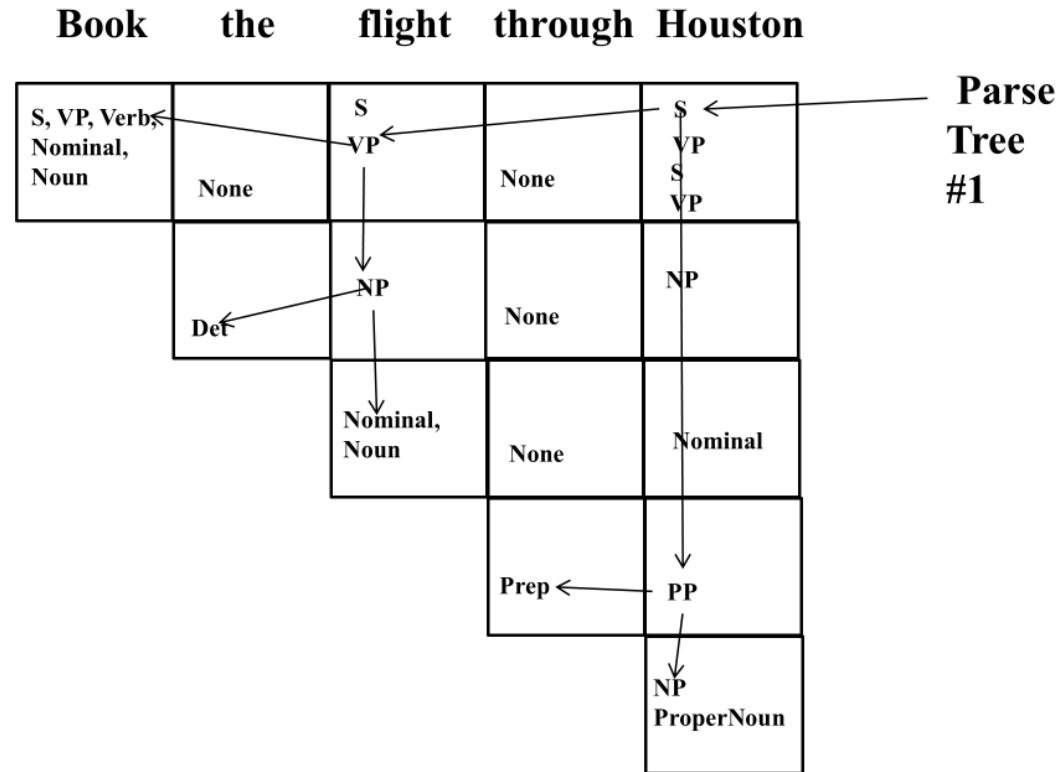
Book the flight through Houston				
S, VP, Verb, Nominal, Noun	None	S VP ←	None	VP S VP
	Det	NP	None	NP
		Nominal, Noun	None	Nominal
			Prep	PP
				NP ProperNoun

CKY Parser

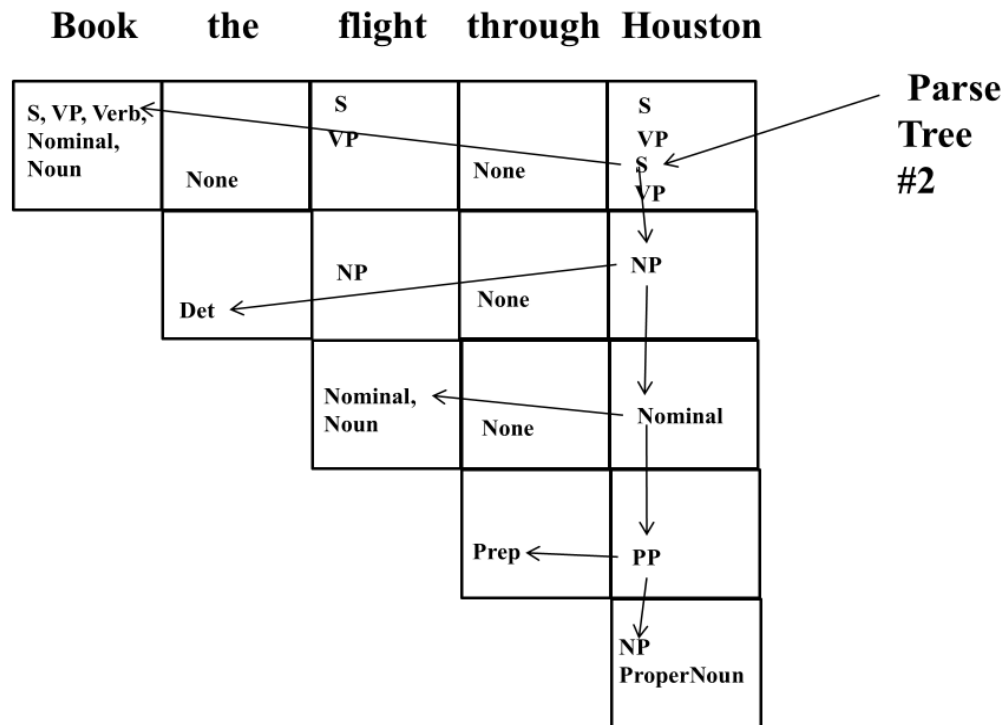
Book the flight through Houston

S, VP, Verb, Nominal, Noun	None	S VP ←	None	S VP S VP
	Det	NP	None	NP
		Nominal, Noun	None	Nominal
			Prep	↓ PP
				NP ProperNoun

CKY Parser



CKY Parser



Probabilistic Context Free Grammar(PCFG)

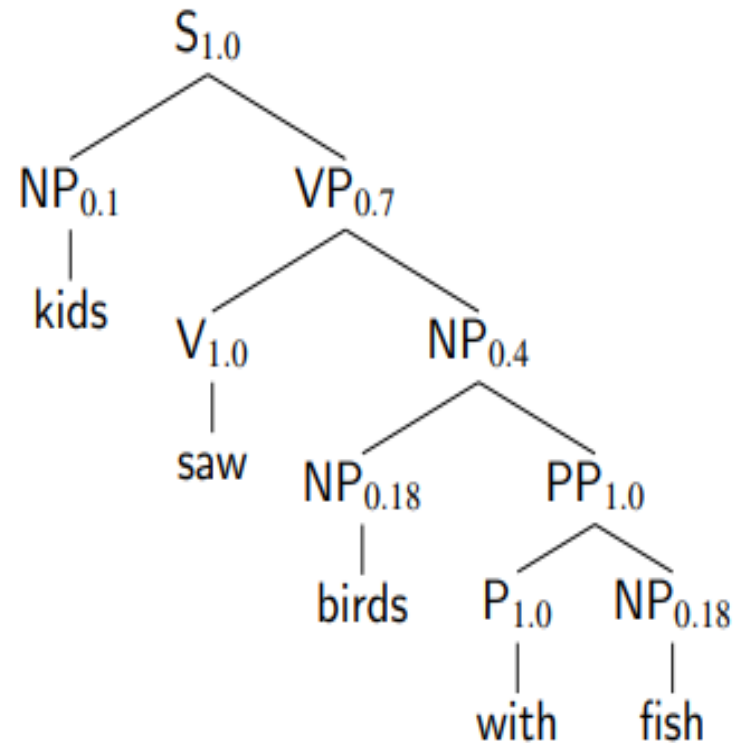


- A PCFG is a probabilistic version of a CFG where each production has a probability.
- Probabilities of all productions rewriting a given non-terminal must add to 1, defining a distribution for each non-terminal.
- String generation is now probabilistic where production probabilities are used to non-deterministically select a production for rewriting a given non-terminal.

$S \rightarrow NP VP$	1.0	$NP \rightarrow NP PP$	0.4
$PP \rightarrow P NP$	1.0	$NP \rightarrow kids$	0.1
$VP \rightarrow V NP$	0.7	$NP \rightarrow birds$	0.18
$VP \rightarrow VP PP$	0.3	$NP \rightarrow saw$	0.04
$P \rightarrow with$	1.0	$NP \rightarrow fish$	0.18
$V \rightarrow saw$	1.0	$NP \rightarrow binoculars$	0.1

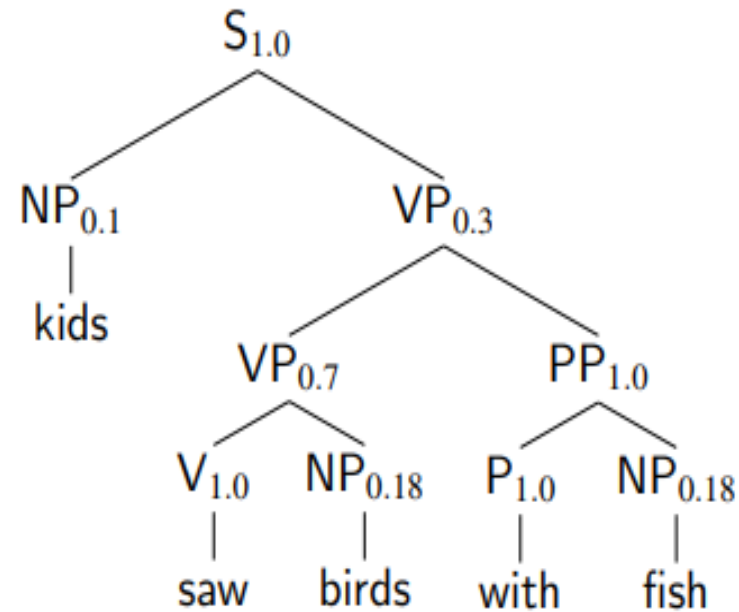
We want to parse “kids saw birds with fish”

Probability of parse 1



- $P(t_1) = 1.0 \cdot 0.1 \cdot 0.7 \cdot 1.0 \cdot 0.4 \cdot 0.18 \cdot 1.0 \cdot 1.0 \cdot 0.18 = 0.0009072$

Probability of parse 2



$$P(t_2) = 1.0 \cdot 0.1 \cdot 0.3 \cdot 0.7 \cdot 1.0 \cdot 0.18 \cdot 1.0 \cdot 1.0 \cdot 0.18 = 0.0006804$$

$P(t_2)$ is less than $P(t_1) = 0.0009072$, so t_1 is preferred.

The probability of a sentence

Since t implicitly includes the words $\vec{w}$, we have $P(t) = P(t, \vec{w})$.

So, we also have a language model. Sentence probability is obtained by summing over

$T(\vec{w})$, the set of valid parses of $\vec{w}$:

$$P(\vec{w}) = \sum_{t \in T(\vec{w})} P(t, \vec{w}) = \sum_{t \in T(\vec{w})} P(t)$$

In our example, $P(\text{kids saw birds with fish}) = 0.0006804 + 0.0009072$.

Three Useful PCFG Tasks

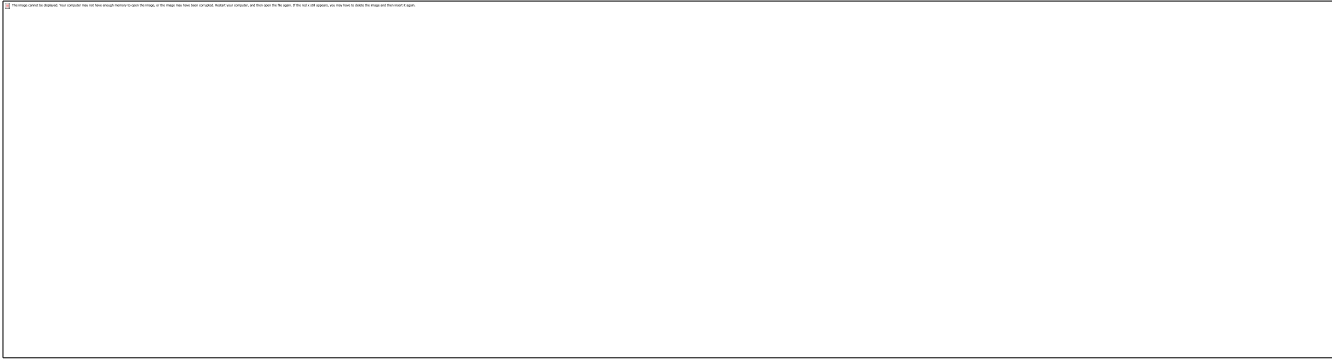
- **Observation likelihood:** To classify and order sentences.
- **Most likely derivation:** To determine the most likely parse tree for a sentence.
- **Maximum likelihood training:** To train a PCFG to fit empirical training data.

Term Frequency-Inverse Document Frequency (TF-IDF)



Term Frequency (TF)

It is a measure of how frequently a term, t , appears in a document, d :



Here, in the numerator, n is the number of times the term “ t ” appears in the document “ d ”. Thus, each document and term would have its own TF value.

We will again use the same vocabulary we had built in the Bag-of-Words model to show how to calculate the TF for Review #2:

Review 2: This movie is not scary and is slow

Here,

- Vocabulary: 'This', 'movie', 'is', 'very', 'scary', 'and', 'long', 'not', 'slow', 'spooky', 'good'
- Number of words in Review 2 = 8
- TF for the word 'this' = (number of times 'this' appears in review 2)/(number of terms in review 2) = $1/8$

Similarly,

- $TF('movie') = 1/8$
- $TF('is') = 2/8 = 1/4$
- $TF('very') = 0/8 = 0$
- $TF('scary') = 1/8$
- $TF('and') = 1/8$
- $TF('long') = 0/8 = 0$
- $TF('not') = 1/8$
- $TF('slow') = 1/8$
- $TF('spooky') = 0/8 = 0$
- $TF('good') = 0/8 = 0$

We can calculate the term frequencies for all the terms and all the reviews in this manner

Term	Review 1	Review 2	Review 3	TF (Review 1)	TF (Review 2)	TF (Review 3)
This	1	1	1	1/7	1/8	1/6
movie	1	1	1	1/7	1/8	1/6
is	1	2	1	1/7	1/4	1/6
very	1	0	0	1/7	0	0
scary	1	1	0	1/7	1/8	0
and	1	1	1	1/7	1/8	1/6
long	1	0	0	1/7	0	0
not	0	1	0	0	1/8	0
slow	0	1	0	0	1/8	0
spooky	0	0	1	0	0	1/6
good	0	0	1	0	0	1/6

Inverse Document Frequency (IDF)

IDF is a measure of how important a term is. We need the IDF value because computing just the TF alone is not sufficient to understand the importance of words:

$$idf_t = \log \frac{\text{number of documents}}{\text{number of documents with term 't'}}$$

We can calculate the IDF values for the all the words in Review 2.

IDF('this') = $\log(\text{number of documents} / \text{number of documents containing the word 'this'})$
 $= \log(3/3) = \log(1) = 0$

Similarly,

- $IDF('movie',) = \log(3/3) = 0$
- $IDF('is') = \log(3/3) = 0$
- $IDF('not') = \log(3/1) = \log(3) = 0.48$
- $IDF('scary') = \log(3/2) = 0.18$
- $IDF('and') = \log(3/3) = 0$
- $IDF('slow') = \log(3/1) = 0.48$

We can calculate the IDF values for each word like this. Thus, the IDF values for the entire vocabulary would be:

Term	Review 1	Review 2	Review 3	IDF
This	1	1	1	0.00
movie	1	1	1	0.00
is	1	2	1	0.00
very	1	0	0	0.48
scary	1	1	0	0.18
and	1	1	1	0.00
long	1	0	0	0.48
not	0	1	0	0.48
slow	0	1	0	0.48
spooky	0	0	1	0.48
good	0	0	1	0.48

Hence, we see that words like “is”, “this”, “and”, etc., are reduced to 0 and have little importance; while words like “scary”, “long”, “good”, etc. are words with more importance and thus have a higher value.

We can now compute the TF-IDF score for each word in the corpus.

Words with a higher score are more important, and those with a lower score are less important:



We can now calculate the TF-IDF score for every word in Review 2:

$$\text{TF-IDF}(\text{'this'}, \text{Review 2}) = \text{TF}(\text{'this'}, \text{Review 2}) * \text{IDF}(\text{'this'}) = 1/8 * 0 = 0$$

Similarly,

- $\text{TF-IDF}(\text{'movie'}, \text{Review 2}) = 1/8 * 0 = 0$
- $\text{TF-IDF}(\text{'is'}, \text{Review 2}) = 1/4 * 0 = 0$
- $\text{TF-IDF}(\text{'not'}, \text{Review 2}) = 1/8 * 0.48 = 0.06$
- $\text{TF-IDF}(\text{'scary'}, \text{Review 2}) = 1/8 * 0.18 = 0.023$
- $\text{TF-IDF}(\text{'and'}, \text{Review 2}) = 1/8 * 0 = 0$
- $\text{TF-IDF}(\text{'slow'}, \text{Review 2}) = 1/8 * 0.48 = 0.06$

Similarly, we can calculate the TF-IDF scores for all the words with respect to all the reviews:

Term	Review 1	Review 2	Review 3	IDF	TF-IDF (Review 1)	TF-IDF (Review 2)	TF-IDF (Review 3)
This	1	1	1	0.00	0.000	0.000	0.000
movie	1	1	1	0.00	0.000	0.000	0.000
is	1	2	1	0.00	0.000	0.000	0.000
very	1	0	0	0.48	0.068	0.000	0.000
scary	1	1	0	0.18	0.025	0.022	0.000
and	1	1	1	0.00	0.000	0.000	0.000
long	1	0	0	0.48	0.068	0.000	0.000
not	0	1	0	0.48	0.000	0.060	0.000
slow	0	1	0	0.48	0.000	0.060	0.000
spooky	0	0	1	0.48	0.000	0.000	0.080
good	0	0	1	0.48	0.000	0.000	0.080

TF-IDF also gives larger values for less frequent words and is high when both IDF and TF values are high i.e the word is rare in all the documents combined but frequent in a single document

1. Bag of Words just creates a set of vectors containing the count of word occurrences in the document (reviews), while the TF-IDF model contains information on the more important words and the less important ones as well. Converting words to numbers with no semantic information.

2. Bag of Words vectors are easy to interpret. However, TF-IDF usually performs better in machine learning models. It is also converting the words to numbers or vectors with some weighted information

Information extraction, relation extraction



- This process of information extraction (IE) turns the unstructured information embedded in texts into structured data.
- More concretely, information extraction is an effective way to populate the contents of a relational database.
- The first step in most IE tasks is to detect and classify all the proper names mentioned in a text — a task generally referred to as named entity recognition NAMED ENTITY RECOGNITION (NER)

- This process involves identifying and pulling out specific pieces of data, such as names, dates, relationships, and more, to transform vast amounts of text into useful, organized information.

Importance of Information Extraction

1. Enhancing Data Usability
2. Automating Data Processing
3. Supporting Decision-Making

Key Components of Information Extraction



Named Entity Recognition (NER)

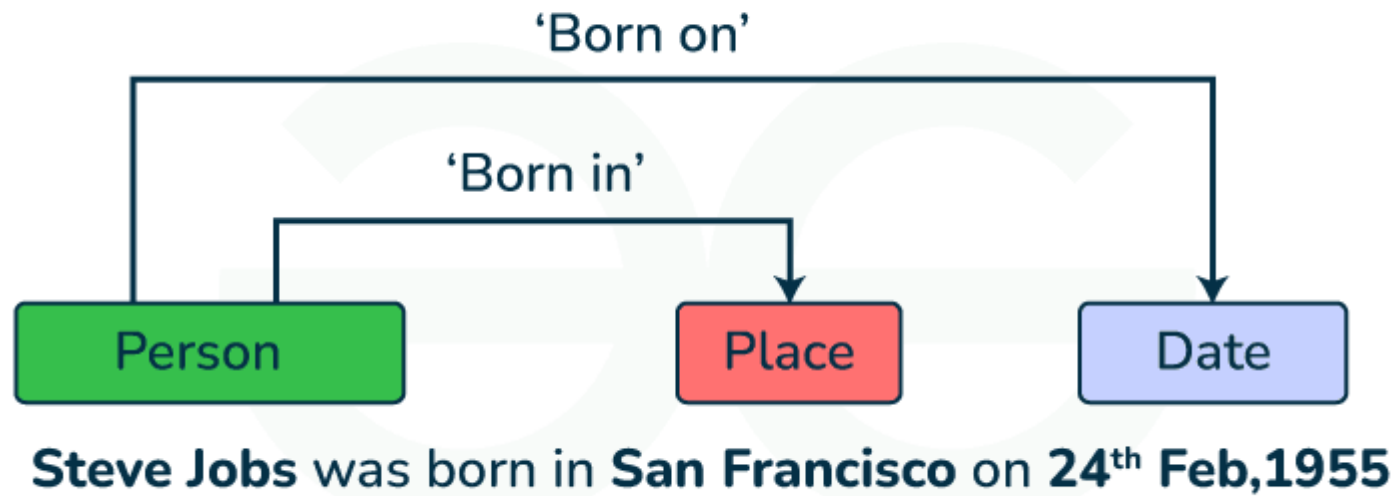
Relationship Extraction

Event Extraction

Relationship Extraction in NLP



- Relationship extraction in natural language processing (NLP) is a technique that helps understand the connections between entities mentioned in text.
- Relationship extraction is an effective technique for organizing information, constructing knowledge graphs, aiding information retrieval, and supporting decision-making processes by identifying and classifying the associations between entities
- The main goal of relationship extraction is to extract valuable insights from text.



Approaches of Extracting Relationships in NLP

1. Rule Based or Pattern Based Approach
2. Supervised Relationship Extraction
3. Unsupervised Relationship Extraction

Rule Based or Pattern Based Approach

- Rule based relationship extraction define rules based on syntactic or semantic structures in text to identify relationships.
- In this grammatical structure of sentence is analyzed to identify dependencies between words.
- Relationships can be inferred based on the syntactic relationships between the entities.

• For Example

CITY **COUNTRY**
Paris is in France

NNP **VBZ** **IN** **NNP**
Paris is in France

Supervised Relationship Extraction



- The task is treated as a classification problem.
- The process involves annotating a training corpus with both entities and their corresponding relations.
- The primary objective is to leverage the annotated texts in the training corpus to teach the classifier to recognize and categorize relationships between pairs of named entities.
- a diverse set of supervised techniques can be employed, ranging from traditional methods like logistic regression and random forest to more advanced models such as recurrent neural networks (RNNs), transformers, or neural classifiers.

Unsupervised Relationship Extraction



- Open Information Extraction (Open IE), aims to identify relationships in text without the availability of labeled training data or predefined lists of relations.
- In Open IE, relations are represented as strings of words, typically starting with a verb.
- The objective is to extract these relationships directly from the text without relying on prior knowledge or annotated examples.